

RAG-App

Hybrid Search & Reranking

Technische Dokumentation

Vom Jupyter-Notebook zur Standalone-Applikation

Version 1.0 – Mai 2026

Thomas Jorg

1 Einleitung

Dieses Dokument beschreibt die Architektur, Implementierung und Inbetriebnahme der RAG-App – einer Standalone-Anwendung für Retrieval-Augmented Generation mit Hybrid Search, Cross-Encoder-Reranking und automatischer Query-Reformulierung.

Die Applikation entstand durch Refactoring eines Jupyter-Notebooks, das ursprünglich für eine Vorlesung entwickelt wurde und sich im täglichen Einsatz bewährt hat. Ziel war es, den Workflow zu vereinfachen: kein VS Code mehr starten, kein Notebook mehr ausführen – stattdessen ein Doppelklick auf ein Startskript, und der Browser öffnet sich mit der fertigen Anwendung.

1.1 Designziele

- **Quick Win:** Gradio als Standalone rauslösen, ohne die bewährte Pipeline-Architektur anzufassen.
- **Cross-Platform:** Lauffähig auf macOS und Windows, identische Codebasis.
- **Konfigurations-getrieben:** LLM-Provider, Modelle und API-Keys in einer zentralen YAML-Datei (inspiriert von der Continue-Extension in VS Code).
- **RAM-schonend:** Kein Hintergrund-Daemon. Start bei Bedarf, Beenden nach Gebrauch.
- **Inkrementell erweiterbar:** Dokumenten-Indizierung bleibt vorerst skriptbasiert, kann später integriert werden.

2 Architektur

Die Applikation folgt einer dreischichtigen Architektur: Konfiguration → Backend (Pipeline) → Frontend (UI).

2.1 Projektstruktur

```
rag-app/
├─ config.yaml           ← Provider, Keys, Modelle, Pipeline-Parameter
├─ config_loader.py      ← YAML lesen, validieren, LM-Studio-Modelle live
abfragen
├─ rag_engine.py         ← Pipeline-Klasse (Retrieve, Rerank, Generate,
Reformulate)
├─ app.py                ← Gradio-UI mit Provider-Dropdown + Chat
├─ run.command / run.bat ← Startskripte (macOS / Windows)
├─ .gitignore
├─ models/               ← Lokaler Modell-Cache (Embeddings, Reranker)
├─ chroma_db/            ← Vektordatenbank (ChromaDB)
├─ bm25_index/           ← BM25-Pickle-Index
└─ documents/           ← Quelldokumente
```

2.2 Datenfluss

Der Datenfluss von der Benutzereingabe bis zur Antwort verläuft wie folgt:

```

Benutzerfrage (Gradio-Chat)
↓
app.py: _extract_text() → reinen String extrahieren
↓
rag_engine.query(question)
↓
LangGraph-Workflow:
    retrieve → rerank → [check_relevance] → generate
                        ↓ (nicht relevant)
                        reformulate → retrieve (Zyklus)
↓
rag_engine.format_result() → Markdown mit Quellen
↓
Gradio-Chat: Antwort anzeigen

```

2.3 Pipeline-Komponenten

Komponente	Aufgabe	Technologie
Hybrid Search	Semantische + lexikalische Suche parallel	ChromaDB + rank_bm25 mit RRF
Reranking	Kandidaten mit Cross-Encoder neu bewerten	sentence-transformers (mmarco)
Relevanz-Gate	Bedingte Kante: relevant → generate, sonst reformulate	LangGraph conditional edges
Query-Reformulierung	Frage umformulieren bei schlechten Ergebnissen	LLM (max. 3 Versuche)
Antwortgenerierung	Antwort auf Basis der besten Chunks erzeugen	LLM via OpenAI-kompatible API

3 Konfiguration (config.yaml)

Die gesamte Konfiguration der App liegt in einer einzigen YAML-Datei. Die Struktur orientiert sich an der Continue-Extension für VS Code und ist bewusst flach und lesbar gehalten.

3.1 Provider-Profile

Jeder LLM-Provider wird als eigenes Profil definiert mit API-Base-URL, API-Key und einer Liste verfügbarer Modelle:

```
providers:
  deepinfra:
    display_name: "DeepInfra"
    api_base: "https://api.deepinfra.com/v1/openai"
    api_key: "sk-..."
    models:
      - meta-llama/Llama-3.3-70B-Instruct-Turbo
      - Qwen/Qwen3-235B-A22B

  lmstudio:
    display_name: "LM Studio (lokal)"
    api_base: "http://localhost:1234/v1"
    api_key: ""
    models: []      # wird zur Laufzeit via /v1/models abgefragt
```

Für LM Studio bleibt die Modellliste leer – der `config_loader` fragt den lokalen Server dynamisch ab und befüllt das Dropdown in der UI automatisch.

3.2 RAG-Parameter

Parameter	Default	Beschreibung
retriever_k	15	Kandidaten pro Retriever (Vektor + BM25)
ensemble_weights	[0.5, 0.5]	Gewichtung [Vektor, BM25] für RRF
rerank_top_n	8	Top-N Chunks nach Cross-Encoder-Reranking
relevance_threshold	0.3	Mindest-Score für das Relevanz-Gate
max_retries	3	Maximale Query-Reformulierungen

4 Komponenten im Detail

4.1 config_loader.py

Verantwortlich für das Laden, Validieren und Bereitstellen der Konfiguration.

- **Automatische Suche:** Findet config.yaml im Skript-Verzeichnis und bis zu 5 Ebenen aufwärts.

- **Deep Merge:** Fehlende Felder werden mit sinnvollen Defaults aufgefüllt – eine minimale YAML genügt.
- **Dynamische Modellabfrage:** Für Provider mit leerer Modellliste (z.B. LM Studio) wird der /v1/models-Endpoint abgefragt. Bei Timeout gibt es eine Warnung statt eines Crashes.
- **Pfadauflösung:** Alle Pfade werden relativ zum Projektverzeichnis via pathlib.Path aufgelöst – Cross-Platform-kompatibel.

4.2 rag_engine.py

Kapselt die gesamte RAG-Pipeline als wiederverwendbare Klasse:

- **RAGEngine.__init__():** Lädt Embedding-Modell (E5), ChromaDB, BM25-Index (rank_bm25 direkt) und Cross-Encoder. Alles parametrisiert über AppConfig.
- **set_llm():** Konfiguriert das LLM zur Laufzeit. Kann beliebig oft aufgerufen werden, ohne die Pipeline neu aufzubauen.
- **_hybrid_retrieve():** Eigene Reciprocal Rank Fusion. Ruft ChromaDB.similarity_search() und rank_bm25 direkt auf – ohne LangChain-Retriever-Abstraktion (s. Abschnitt 6).
- **query() / format_result():** Trennung von Rohzugriff und Markdown-Aufbereitung.

4.3 app.py

Das Gradio-6-kompatible Frontend mit folgenden Elementen:

- **Provider-Dropdown:** Befüllt aus config.yaml. Bei Wechsel wird die Modell-Liste dynamisch aktualisiert.
- **Modell-Dropdown:** Erlaubt auch manuelle Eingabe (allow_custom_value) für Modelle, die nicht in der Config stehen.
- **Temperature-Slider:** 0.0 bis 1.5, Default aus Config.
- **Chat-Bereich:** Messages-Format (Gradio 6). Zwei-Schritt-Callback: user_message (sofort) → bot_response (Generator).
- **Pipeline-Details:** Eingeklapptes Accordion mit allen Parametern und Pfaden als Referenz.

5 Cross-Platform-Betrieb

5.1 Dateisystem

Alle Pfade verwenden `pathlib.Path` und werden relativ zum Projektverzeichnis aufgelöst. In der `config.yaml` stehen nur relative Pfade (`./chroma_db`, `./models` etc.). Der BM25-Pickle-Index und die ChromaDB sind plattformneutral.

5.2 Startskripte

Plattform	Datei	Besonderheit
macOS	<code>run.command</code>	Sucht <code>.venv</code> automatisch; Fallback <code>python3</code> → <code>python</code>
Windows	<code>run.bat</code>	Sucht <code>.venv</code> automatisch; hält Fenster bei Fehler offen

Beide Skripte suchen das Virtual Environment zuerst im Projektverzeichnis, dann eine Ebene darüber. Falls es woanders liegt, kann `VENV_PATH` explizit gesetzt werden.

5.3 Backup-Strategie

Die Projektverzeichnisse sind auf Mac und Windows gespiegelt (Teil der bestehenden Backup-Strategie). Da alle Pfade relativ sind und die Config-Datei zum Projekt gehört, funktioniert die App auf beiden Rechnern ohne Anpassung.

6 Gelöste Kompatibilitätsprobleme

Bei der Migration vom Notebook zur App traten mehrere Kompatibilitätsprobleme auf, die hier für die Nachwelt dokumentiert sind.

6.1 Gradio 6: Content-Block-Format

Problem: Gradio 6 speichert Chat-Nachrichten nicht als einfache Strings, sondern als Content-Block-Listen im OpenAI-Format:

```
// Erwartet:
"suche aufgaben zur harmonischen schwingung"

// Tatsächlich von Gradio 6 geliefert:
[{"type": "text", "text": "suche aufgaben zur harmonischen schwingung"}]
```

Lösung: Zwei Sicherungsschichten – `_extract_text()` in `app.py` extrahiert den reinen String vor der Übergabe an die Engine. `_ensure_str()` in `rag_engine.py` fängt Content-Blöcke als zusätzliches Sicherheitsnetz ab.

6.2 Gradio 6: API-Änderungen

Gradio 5	Gradio 6	Grund
----------	----------	-------

theme/css in gr.Blocks()	theme/css in .launch()	Parameter verschoben
show_copy_button=True	buttons=["copy", "copy_all"]	Parameter umbenannt/ersetzt
Chatbot mit Tuple-Format	Chatbot mit Messages-Format (Dicts)	Einziger unterstützter Modus
Chatbot(type=...)	type-Parameter entfernt	Messages ist Standard

6.3 LangChain: EnsembleRetriever inkompatibel

Problem: LangChains `BaseRetriever.invoke()` in neueren Versionen transformiert den Query-Input intern zu einem Dict. Der `EnsembleRetriever` aus `langchain_classic` reicht diesen transformierten Input an die Sub-Retriever weiter, die einen String erwarten.

Lösung: Der `EnsembleRetriever` wurde komplett entfernt. Stattdessen implementiert `_hybrid_retrieve()` die Reciprocal Rank Fusion selbst:

- Vektor-Suche: `ChromaDB.similarity_search()` direkt (umgeht den Retriever)
- BM25-Suche: `rank_bm25.BM25Okapi` direkt (umgeht LangChain komplett)
- Fusion: Eigene RRF-Implementierung mit konfigurierbaren Gewichten

Dieser Ansatz ist nicht nur kompatibel, sondern auch transparenter und besser debugbar als die LangChain-Abstraktion.

6.4 LangChain: response.content als Liste

Problem: Manche LLM-Provider liefern `response.content` als Liste von Content-Blöcken statt als String.

Lösung: `_content_to_str()` in `rag_engine.py` normalisiert die Antwort transparent – String bleibt String, Liste wird zusammengesetzt.

7 Inbetriebnahme

7.1 Voraussetzungen

- Python 3.11+ mit Virtual Environment
- Installierte Pakete: `gradio`, `langchain-community`, `langchain-openai`, `langchain-huggingface`, `langgraph`, `sentence-transformers`, `chromadb`, `rank_bm25`, `pyyaml`
- Bestehende Indizes: `chroma_db/` und `bm25_index/chunks.pkl` (erzeugt durch das Indizierungs-Notebook)
- Mindestens ein konfigurierter LLM-Provider mit API-Key (oder LM Studio lokal)

7.2 Schnellstart

1. Alle Python-Dateien und Startskripte ins Projektverzeichnis kopieren (dort wo `chroma_db/` etc. bereits liegen).
2. API-Keys in `config.yaml` eintragen.

3. Startskript ausführen:

```
# macOS:
chmod +x run.command
open run.command      # oder Doppelklick im Finder

# Windows:
run.bat               # Doppelklick im Explorer

# Alternativ (beide Plattformen):
python app.py
```

4. Der Browser öffnet sich automatisch auf <http://127.0.0.1:7860>.

5. Provider und Modell auswählen, „Aktivieren“ klicken, Fragen stellen.

8 Mögliche Erweiterungen

- **Dokumenten-Upload & Re-Indizierung:** Drag & Drop von PDFs in die UI, automatische Aktualisierung von ChromaDB und BM25-Index.
- **FastAPI-Backend:** Trennung von UI und Pipeline für flexiblere Frontends oder API-Zugriff.
- **Conversation Memory:** Mehrere Fragen im Kontext einer Sitzung, mit Chat-History als zusätzlichem Kontext.
- **Streaming-Antworten:** LLM-Antwort token-weise in der UI anzeigen statt auf die vollständige Antwort zu warten.
- **Desktop-Wrapper:** Tauri oder Electron für eine native App ohne separates Browser-Fenster.